TP 5 : Invocation des méthodes sur des objets distants avec java RMI

Activité 5-1 : Service de conversion de devise

Objectifs

Au terme de cette activité, l'étudiant sera capable de :

- Mettre en œuvre une communication client/serveur via les Remote Method Invocation (RMI) de Java.
- Comprendre le cycle de vie d'un service distant (Définition de l'interface, Implémentation, Enregistrement, Invocation).

Environnement

- JDK >=1.8
- Eclipse IDE for Java developers Ou IntelliJ IDAE > 1.8 OU VS code
- Git, GitHub

Type de l'activité

- Individuel

Ressources

- Consultez le cours « Développement d'applications réparties : RMI ».

Livrables

- Le lien du repository dans l'espace de dépôt de la plateforme ENT
- Le fichier README

Critères d'évaluation :

- Lien partagé avec un TP conforme aux consignes et dans le délai : **5 pts**,
- Lien partagé avec TP incomplet ou partiellement conforme ou hors délais : **2-4 pts**,
- Absence de dépôt ou hors sujet ou en retard d'une période : **0-1 pt**

Enoncé et travail demandé

L'objectif de cet atelier est de concevoir une application répartie fournissant un service de conversion de devises. Le serveur offrira aux clients la possibilité de convertir un montant en euros vers le dinar.

Cette activité comporte cinq étapes :

Étape 1 : Préparation de l'espace de travail

Dans votre espace de travail, créez un répertoire sous le nom « **votreNomPrenom_LSI3_DevAppReparties** ».

Étape 2 : Développement du serveur

1. Créez un nouveau Projet Java nommé **RMIConversion_Server.**
2. Créez un package « **rmiService** » dans lequel on crée l'interface distante « **IConversion** ».
   Cette interface doit définir la méthode distante :
   « **double convertirMontant(double mt)** ».

NB : Importez les packages nécessaires et ajoutez les exceptions appropriées (Remote et RemoteException).

3. Implémentez le service : Dans le même package « rmiService », créez une classe nommée « **ConversionImpl** » et implémentez-y l'interface IConversion.
4. Créez un deuxième package nommé « **rmiServer** ». Dans ce package, créez une classe appelée « **ConversionServer** ». L'application ConversionServer doit :
   i. activer l'annuaire RMIRegistry
   ii. créer une instance de la classe ConversionImpl.
   iii. publier la référence de cet objet dans l'annuaire (RMIRegistry).

Étape 3 : Développement du client

1. Créez un nouveau projet Java nommé « **RMIConversion_Client** ».
2. Exportation l'interface distante depuis le projet serveur :
   i. Depuis le projet RMIConversion_Server, exportez le package rmiService en ne sélectionnant que l'interface **IConversion** (ne pas inclure ConversionImpl).

ii. Générez un fichier **.jar** nommé **conversionClient.jar**, puis enregistrez-le dans votre espace de travail.

3. Importation du .jar dans le projet client

   i. Ajoutez le fichier conversionClient.jar au Classpath du projet RMIConversion_Client :

Procédure : Clic droit sur le projet → Properties → Java Build Path → Libraries → Classpath → Add External JARs... → sélectionner conversionClient.jar → Apply and Close.

4. Créez un package nommé « **rmiClient** », puis ajoutez-y une classe appelée « **ConversionClient** ». L'application RMIConversionClient doit :

   ii. Chercher la référence vers l'objet distant IConversion dans l'annuaire (RMI registry).

   iii. Invoquer la méthode convertirMontant(500) via cette référence.

Étape 4 : Exécution de l'application

Lancez d'abord le serveur RMI, puis exécutez le client afin de vérifier le fonctionnement de l'application.

Étape 5 : Dépôt du livrable

Dans cette dernière étape, vous êtes invités à :

- Déposez votre projet sur GitHub ainsi que le lien du repository dans l'espace de dépôt de la plateforme ENT.
- Le fichier README doit contenir des imprimés écrans accompagnés d'une synthèse.